DAY-20

PYTHON 90 DAYS CHALLENGES

Topics from PDF pages 215–218

Advanced Functions: Recursion, Nested Functions, Closures, Function Annotations, and Higher-Order Functions.

11.6 Recursion

Recursion means a function calling itself.

A correct recursive function must have a stopping condition.

Important points:

• Recursion is used when a problem can be broken into smaller versions of the same problem.

• A recursive function calls itself.

• Every recursion must have a base case.

• The base case stops the recursion.

• Without a base case, recursion continues until Python raises RecursionError.

Terms

Base case: Condition that stops recursion.

Recursive case: Function calling itself.

Example 1: Factorial

Factorial means: 5! = 5 × 4 × 3 × 2 × 1

```
def factorial(number):
    if number == 1:
        return 1
    return number * factorial(number - 1)

print(factorial(5))
```

```
Output: 120
```

*factorial(5) = 5 * factorial(4) = 5 * 4 * factorial(3) = 5 * 4 * 3 * factorial(2) = 5 * 4 * 3 * 2 * factorial(1) = 5 * 4 * 3 * 2 * 1 = 120*

11.7 Nested Functions

A nested function is a function defined inside another function.

Syntax

```
def outer_function():
    def inner_function():
        statement

    inner_function()
```

• A nested function is created inside another function.

• The inner function can be used only inside the outer function.

• Nested functions are useful for hiding helper logic.

• They are also used in closures and decorators.

Example 1

```
def outer():
    print("Outer function started")

    def inner():
        print("Inner function executed")

    inner()

outer()
```

Output:

```
Outer function started
Inner function executed
```

Important point: inner() cannot be called directly outside outer().

11.8 Closures

A closure is created when an inner function remembers variables from its outer function, even after the outer function has finished.

Important points:

• A closure needs a nested function.

• The inner function uses a variable from the outer function.

• The outer function returns the inner function.

• The inner function remembers the outer variable.

• Closures are useful for creating customized functions.

The source PDF shows the closure output as: 10.

11.9 Function Annotations

Function annotations are used to add type hints to function parameters and return values.

Syntax

```
def function_name(parameter: type) -> return_type:
    statement
```

• Function annotations describe expected data types. They make code easier to understand.

• They help editors and tools detect possible mistakes.

• Python does not automatically enforce these types at runtime.

• Annotations are stored in the function's __annotations__ attribute.

Example 1

```
def add(a: int, b: int) -> int:
    return a + b

print(add(10, 20))
```

```
Output: 30
```

```
{'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
```

Important point: Type hints are hints. Python does not stop wrong types automatically.

11.10 Higher-Order Functions

A higher-order function is a function that does at least one of these:

1. Takes another function as an argument.

2. Returns another function.

Explanation:

• Higher-order functions are possible because Python functions are objects.

• They are used in map(), filter(), reduce(), decorators, and callbacks.

• They make code flexible and reusable.

• They are common in functional programming.

Example 1: Function as Argument

```
def shout(text):
    return text.upper()

def process_text(function, text):
    return function(text)

result = process_text(shout, "hello")
print(result)
```

Output: HELLO

Here, process_text() is a higher-order function because it accepts another function.